UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

PROYECTO 02

INTEGRANTES:	**CARNET:**
CRISTHIAN ESTUARDO DE LEÓN PÉREZ	1540225

QUETZALTENANGO, NOVIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S.J

ESTRUCTURA DEL REGISTRO

Para poder implementar el acceso aleatorio de forma eficiente, es indispensable que cada registro (contacto) ocupe exactamente la misma cantidad de bytes dentro del archivo .dat. Esto permite que el sistema calcule matemáticamente la posición de cualquier contacto y salte directamente a él mediante el uso de punteros, sin necesidad de leer el archivo de forma secuencial.

Para lograr esto, se definió una longitud máxima estricta para cada atributo del contacto. Si la información ingresada por el usuario es menor a esta longitud, el sistema rellena el espacio sobrante con caracteres en blanco hacia la derecha.

La estructura asignada para cada registro se compone de la siguiente manera:

- ID del Contacto: 10 bytes (caracteres).
- Nombre: 40 bytes.
- Teléfono: 15 bytes.
- Correo Electrónico: 40 bytes.
- Ciudad: 20 bytes.

La suma de los datos útiles del contacto da un subtotal de 125 bytes. A este valor se le añade 1 byte adicional que corresponde al carácter oculto de salto de línea (\n). Este carácter garantiza que, además de funcionar a nivel de bytes, el archivo de texto sea estructurado y legible si se abre en un editor de texto estándar. Por lo tanto, el tamaño total exacto de cada registro es de 126 bytes.

IMPLEMENTACIÓN DE SEEK() Y TELL()

El acceso aleatorio es el núcleo operativo de la aplicación, permitiendo operaciones de lectura y escritura de forma directa y eficiente sin importar el tamaño del archivo. Para lograr esto, se utilizaron las funciones tell() y seek().

tell() se utilizó para el cálculo de posiciones lógicas. Al momento de agregar un nuevo contacto, el archivo .dat se abre en modo append ("a"). Antes de escribir la cadena del nuevo registro, se invoca la función f.tell() para obtener la posición exacta del puntero en bytes al final del archivo. Al dividir este valor entero entre el tamaño fijo definido (126 bytes), se obtiene el número de registro lógico (posición indexada), el cual se retorna para ser almacenado en el índice JSON.

```
def agregar_contacto(id_contacto, nombre, telefono, correo, ciudad):
    registro = empaquetar_contacto(id_contacto, nombre, telefono, correo,
ciudad)
    with open(RUTA_ARCHIVO, "a", encoding="utf-8") as f:
        byte_actual = f.tell()
        f.write(registro)
    numero_registro = byte_actual // TAMANO_TOTAL_REGISTRO
    return int(numero_registro)
```

Para consultar o editar un contacto existente sin tener que recorrer el archivo secuencialmente, se recupera su número de registro desde el índice JSON. Al multiplicar este número lógico por 126, se calcula la coordenada del byte exacto donde inicia la información de dicho contacto.

Posteriormente, se utiliza f.seek(posicion_byte) para desplazar el puntero del sistema operativo directamente a esa ubicación física. Durante la modificación, el archivo se abre en modo lectura/escritura ("r+"), lo que permite sobrescribir únicamente los 126 bytes correspondientes al contacto editado, logrando una actualización que mantiene intacta la integridad del resto del archivo.

```
def modificar_contacto(numero_registro, id_contacto, nombre, telefono,
correo, ciudad):
    if not os.path.exists(RUTA_ARCHIVO):
        return False
    registro = empaquetar_contacto(id_contacto, nombre, telefono, correo,
ciudad)
    posicion_byte = numero_registro * TAMANO_TOTAL_REGISTRO
    with open(RUTA_ARCHIVO, "r+", encoding="utf-8", newline='\n') as f:
        f.seek(posicion_byte)
        f.write(registro)
    return True
```

FUNCIONAMIENTO DEL ÍNDICE

El sistema implementa un índice auxiliar estructurado en formato JSON (indice.json). Su función exclusiva es mapear el ID único de cada contacto (la clave) con su respectivo número de registro lógico (el valor) dentro del archivo .dat principal.

Cuando un usuario solicita buscar, cargar o modificar un contacto por su ID, el flujo de ejecución es el siguiente:

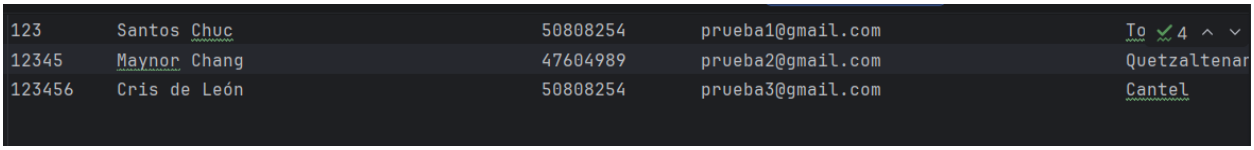
1. El sistema consulta el índice en memoria para verificar si la clave (ID) existe.
2. Si el ID es encontrado, el índice devuelve de manera instantánea el número de registro asociado.
3. El backend toma ese número, lo multiplica por 126 bytes, y ejecuta un único salto con seek() para leer exclusivamente esa porción del archivo en el disco duro.

¿Por qué es más eficiente que una búsqueda secuencial? En una búsqueda secuencial tradicional, el programa está obligado a abrir el archivo .dat y leerlo desde el inicio (byte cero). Debe cargar a la memoria el primer bloque de 126 bytes, desempaquetarlo, extraer el ID y compararlo. Si no coincide, debe avanzar al siguiente bloque, repitiendo este proceso cíclicamente hasta hallar una coincidencia o llegar al final del archivo. Este método requiere múltiples y costosas operaciones de entrada/salida en el disco de almacenamiento, resultando en una complejidad de tiempo lineal $O(n)$.

Por el contrario, la búsqueda mediante un índice reduce drásticamente las operaciones de disco. Al utilizar un diccionario cargado en RAM, la búsqueda de la clave tiene un costo computacional de tiempo constante $O(1)$. Independientemente de si la base de datos contiene diez contactos o un millón, el sistema solo necesita una única operación de lectura en el disco duro para extraer la información, logrando un rendimiento óptimo y escalable.

VERIFICACIÓN DE INTEGRIDAD

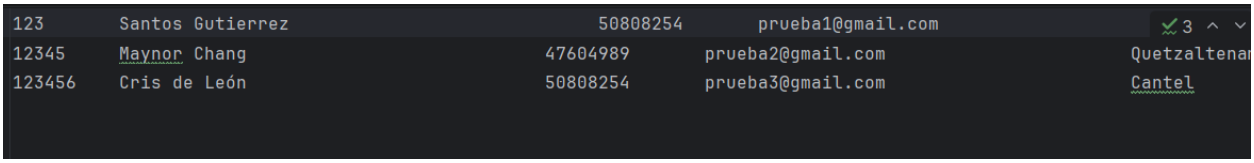
Para hacer la verificación de integridad primero se debe de tener un archivo con datos, como se puede ver en la imagen.



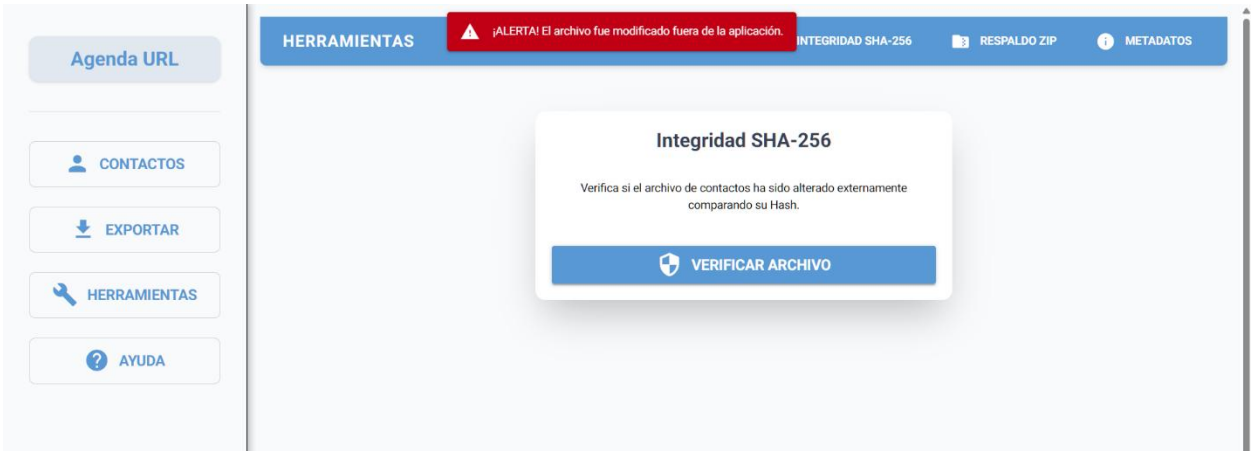
Cuando se tenga el archivo listo se selecciona la opción de “INTEGRIDAD SHA-256” para hacer la verificación. En este caso se puede ver que no se ha alterado el archivo.



Ahora en el siguiente ejemplo se modificó el archivo de los contactos para realizar una nueva verificación (se cambió el apellido de Santos).



Al realizar nuevamente la verificación sale que fue modificado fuera de la aplicación.

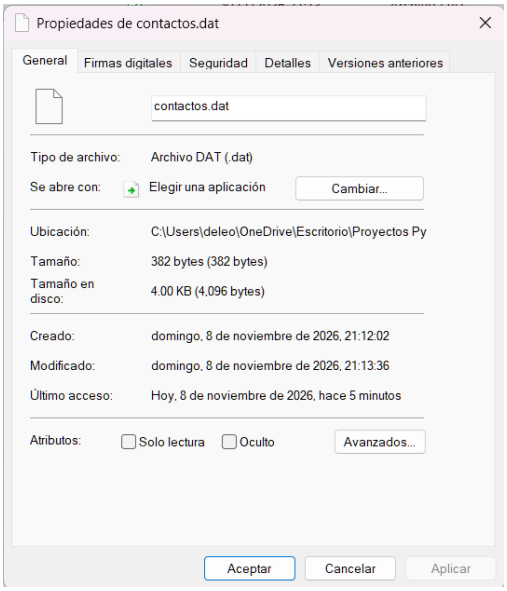


COMPARACIÓN DE TAMAÑO ENTRE ARCHIVOS

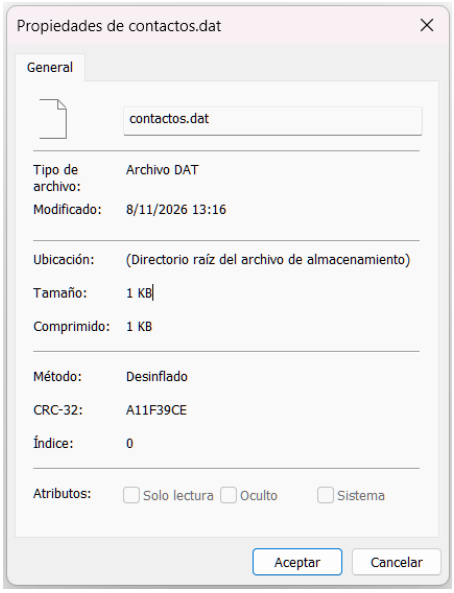
Como medida de seguridad adicional y optimización de almacenamiento, la aplicación incluye una herramienta para generar un respaldo comprimido (respaldo_agenda.zip) que contiene tanto la base de datos principal (contactos.dat) como su índice asociado (indice.json).

Al utilizar el algoritmo de compresión ZIP_DEFLATED, se logra reducir significativamente el espacio en disco que ocupan los datos. A continuación, se presenta una comparación de los tamaños de los archivos antes y después de la compresión, demostrando la eficiencia del respaldo:

Original



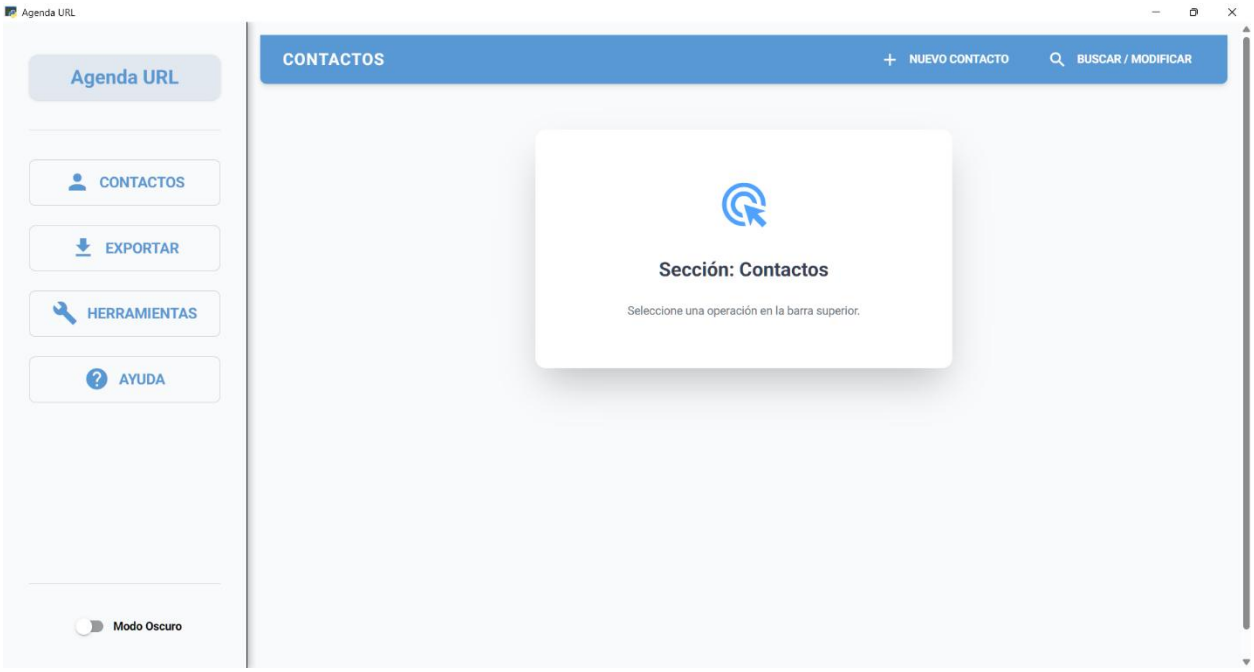
ZIP

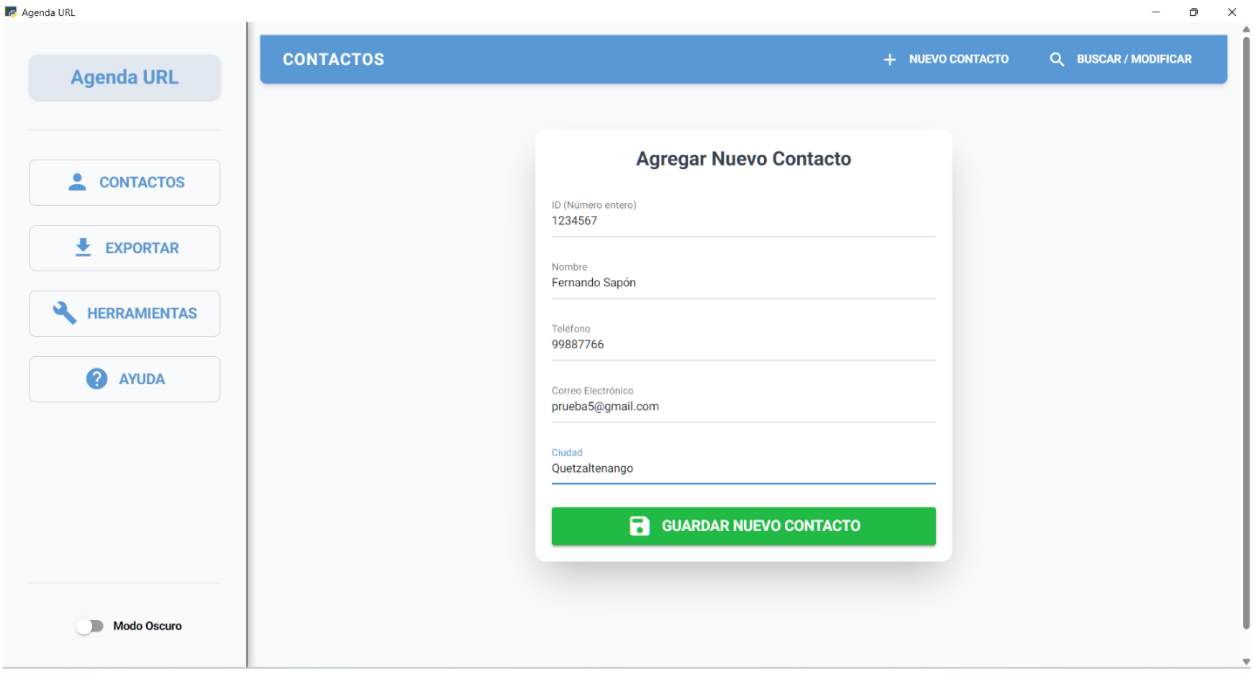


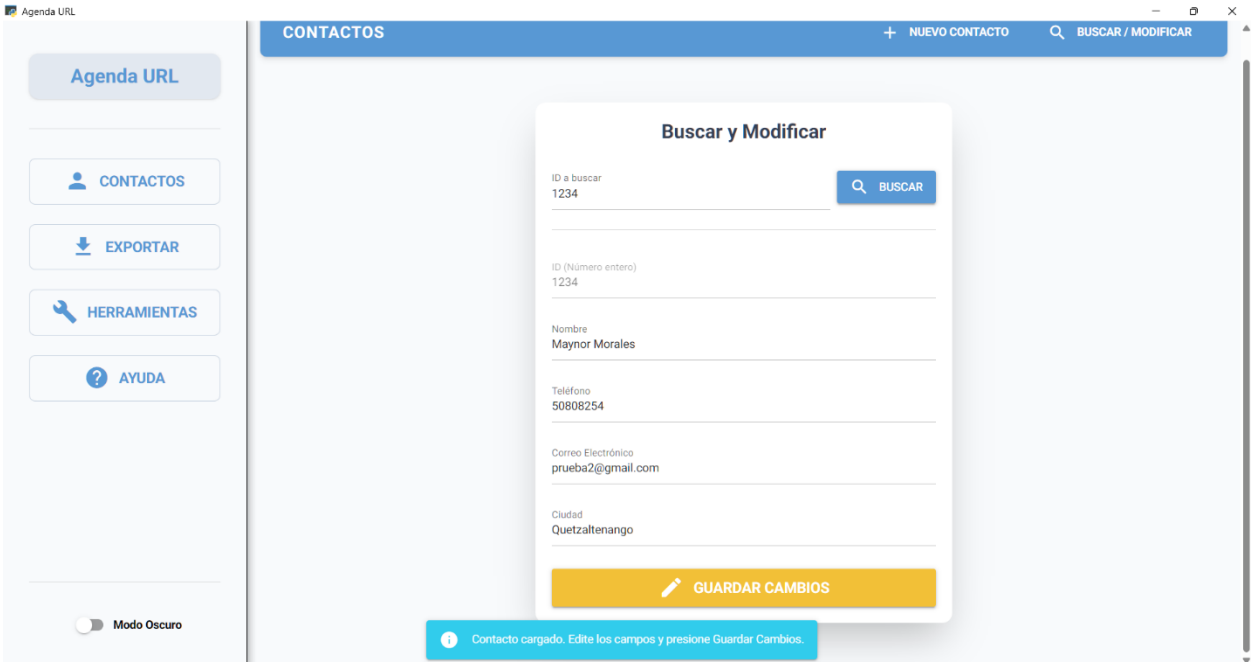
Como se puede observar en la evidencia, la compresión ZIP resulta ser un método eficaz para almacenar respaldos históricos de la agenda, minimizando el impacto en la capacidad de almacenamiento del sistema sin comprometer la disponibilidad de la información.

CAPTURAS GENERALES DEL PROGRAMA

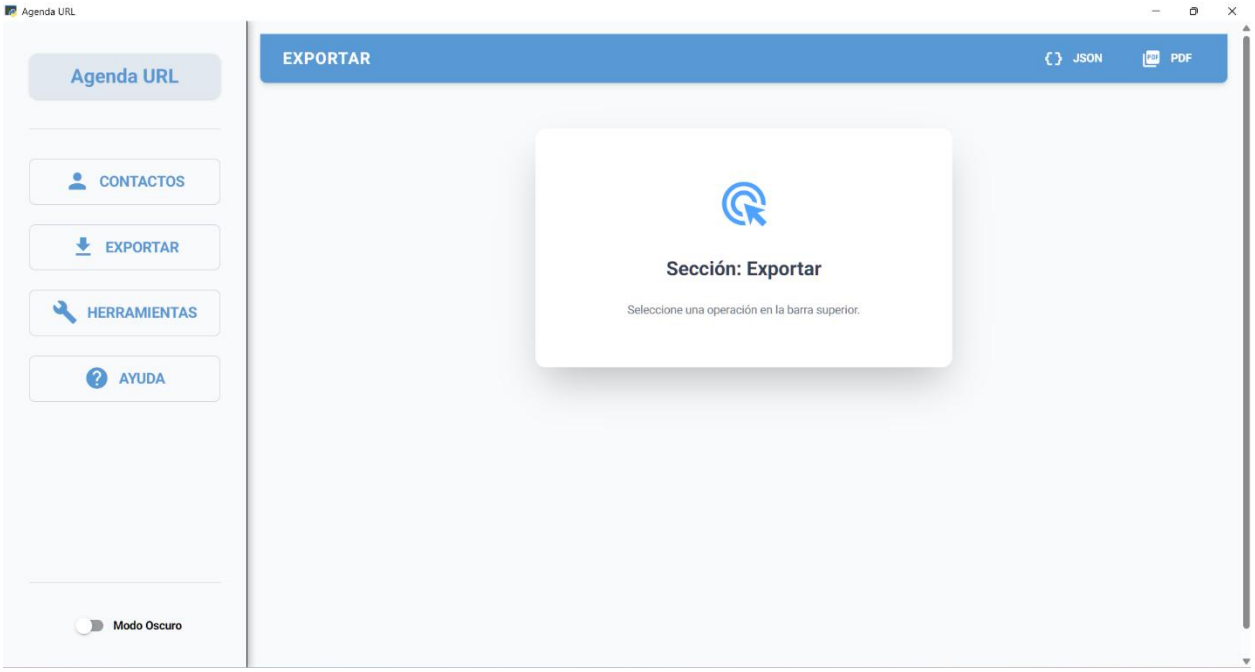
Opción de contactos:

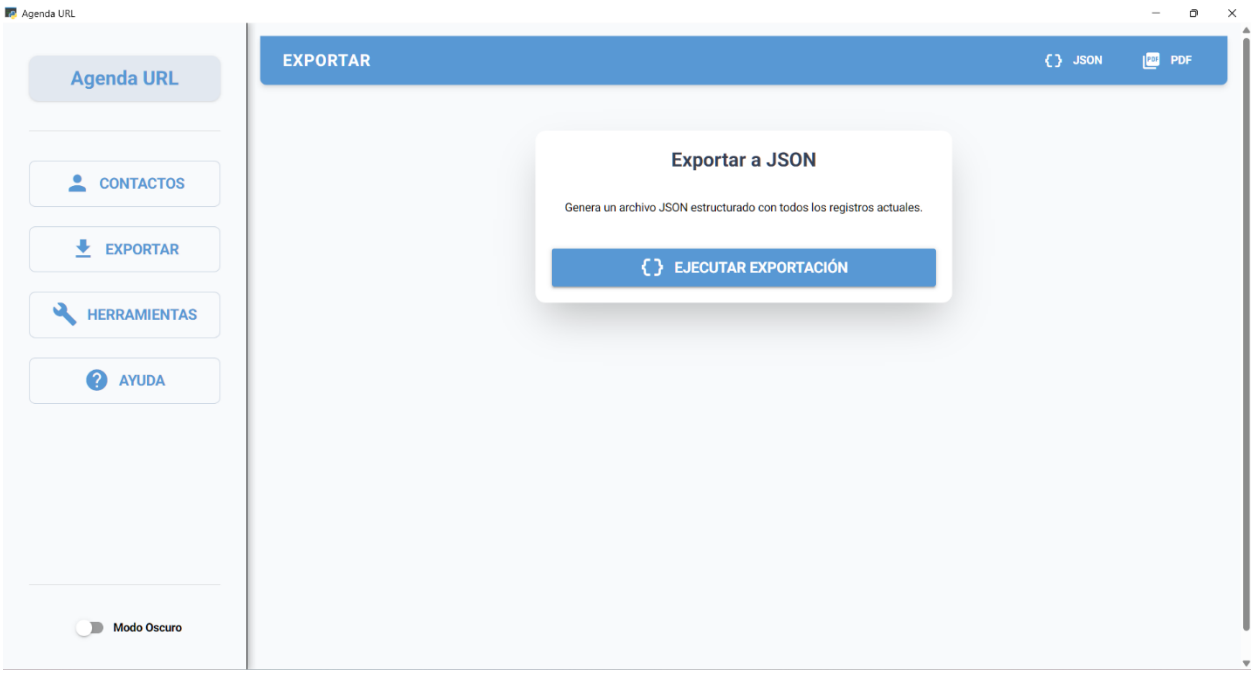


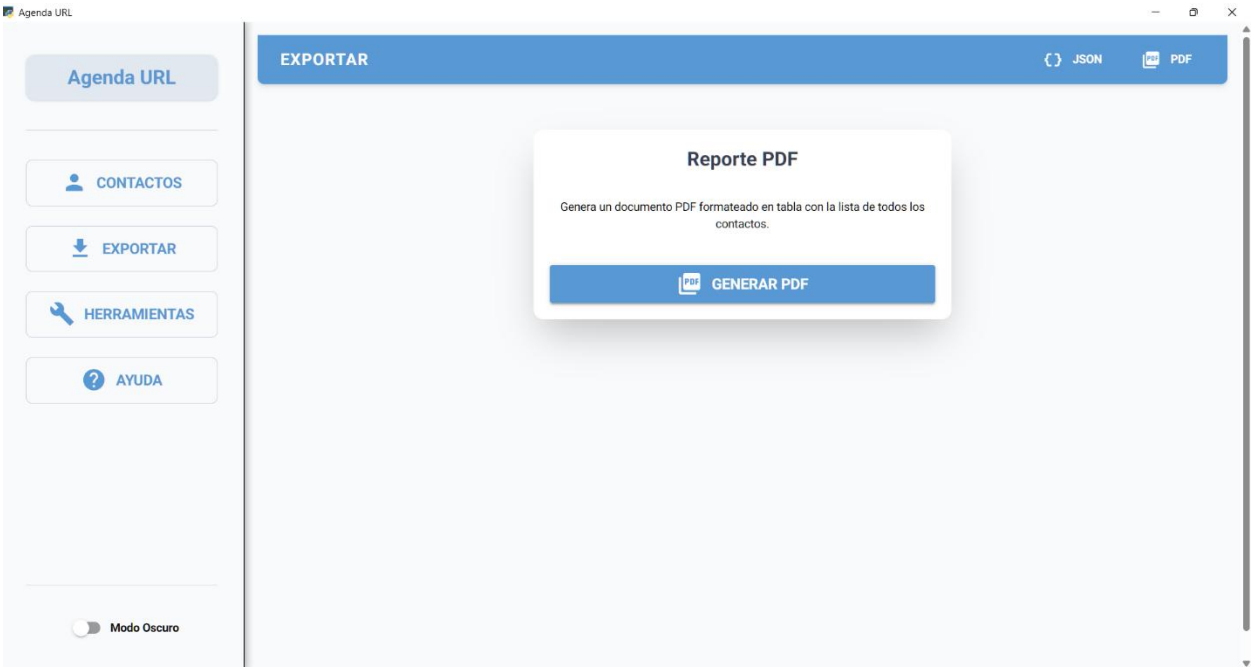


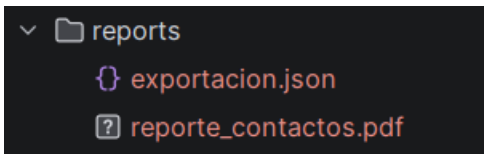


Opción de exportar:

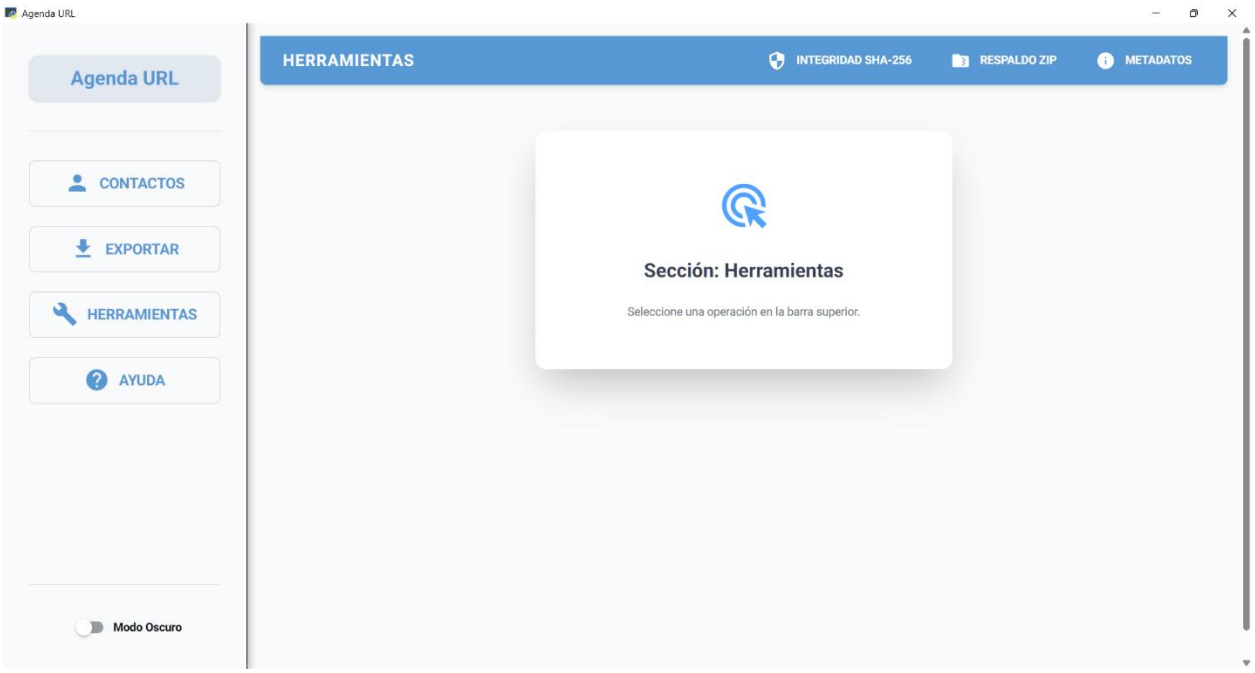


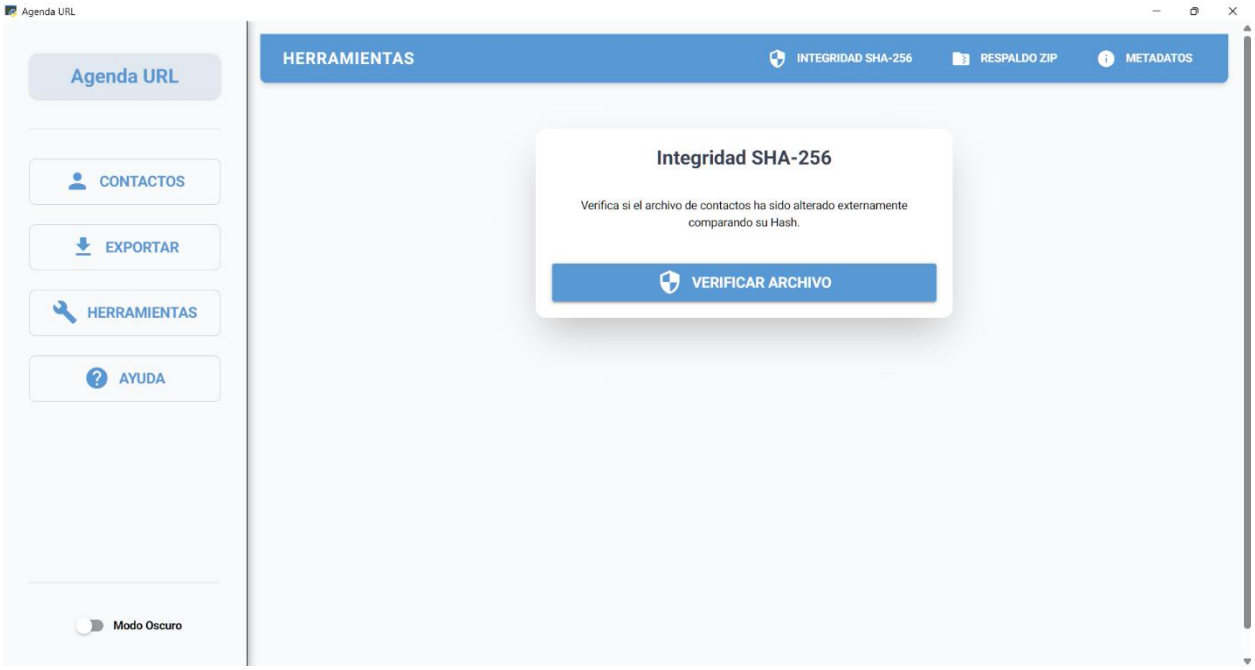

Agenda URL
Agenda URL
CONTACTOS
EXPORTAR
HERRAMIENTAS
AYUDA
Modo Oscuro
EXPORTAR
JSON
PDF
Exportar a JSON
Genera un archivo JSON estructurado con todos los registros actuales.
EJECUTAR EXPORTACIÓN

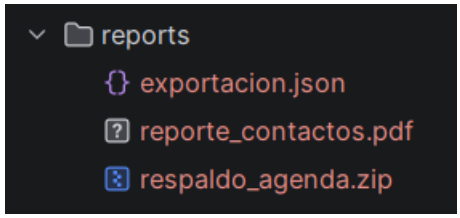

Agenda URL
Agenda URL
CONTACTOS
EXPORTAR
HERRAMIENTAS
AYUDA
Modo Oscuro
EXPORTAR
JSON
PDF
Reporte PDF
Genera un documento PDF formateado en tabla con la lista de todos los contactos.
GENERAR PDF

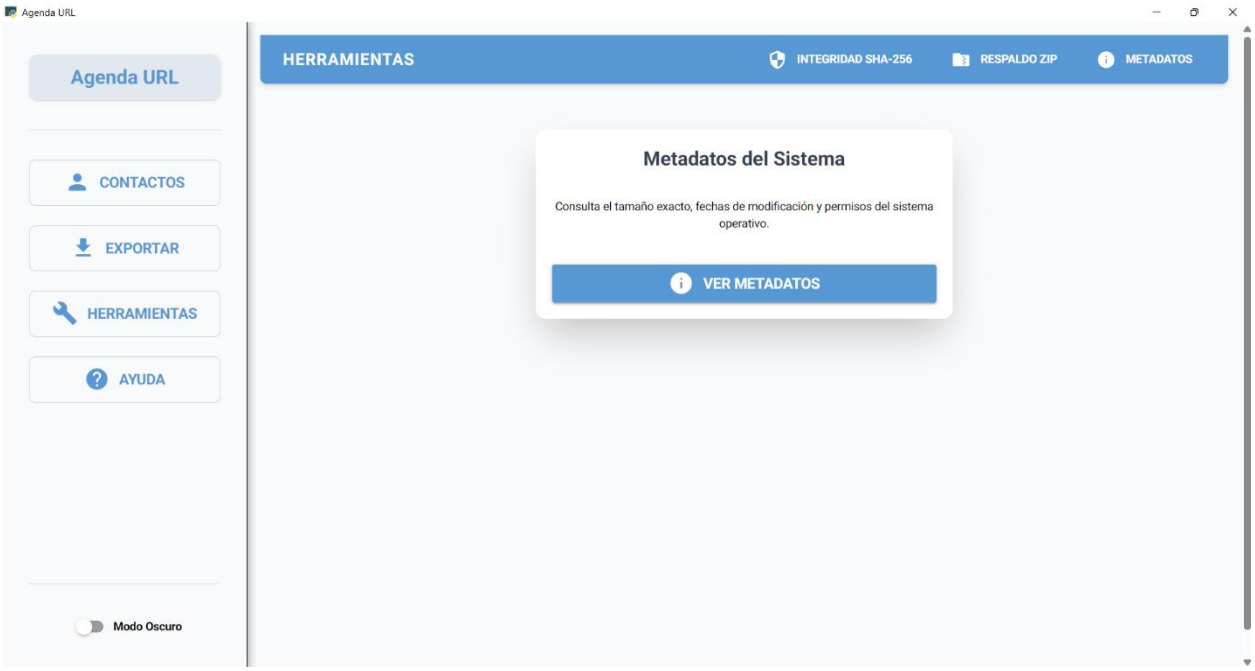

reports
exportacion.json
reporte_contactos.pdf

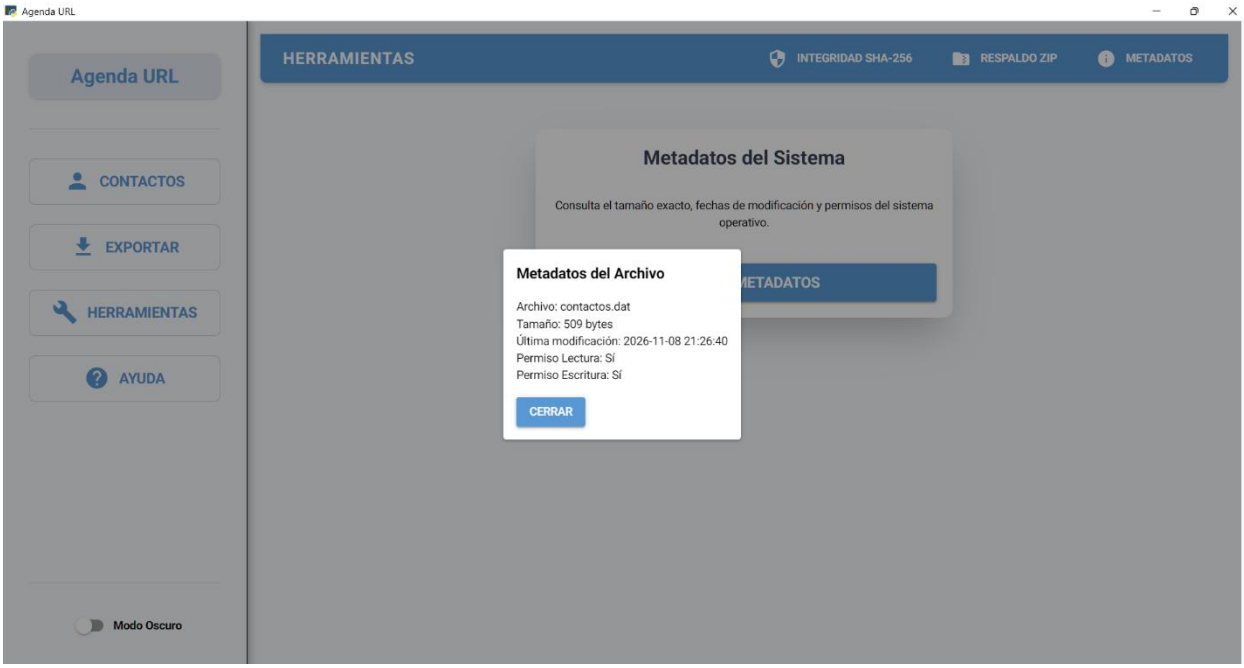
Opción de herramientas:



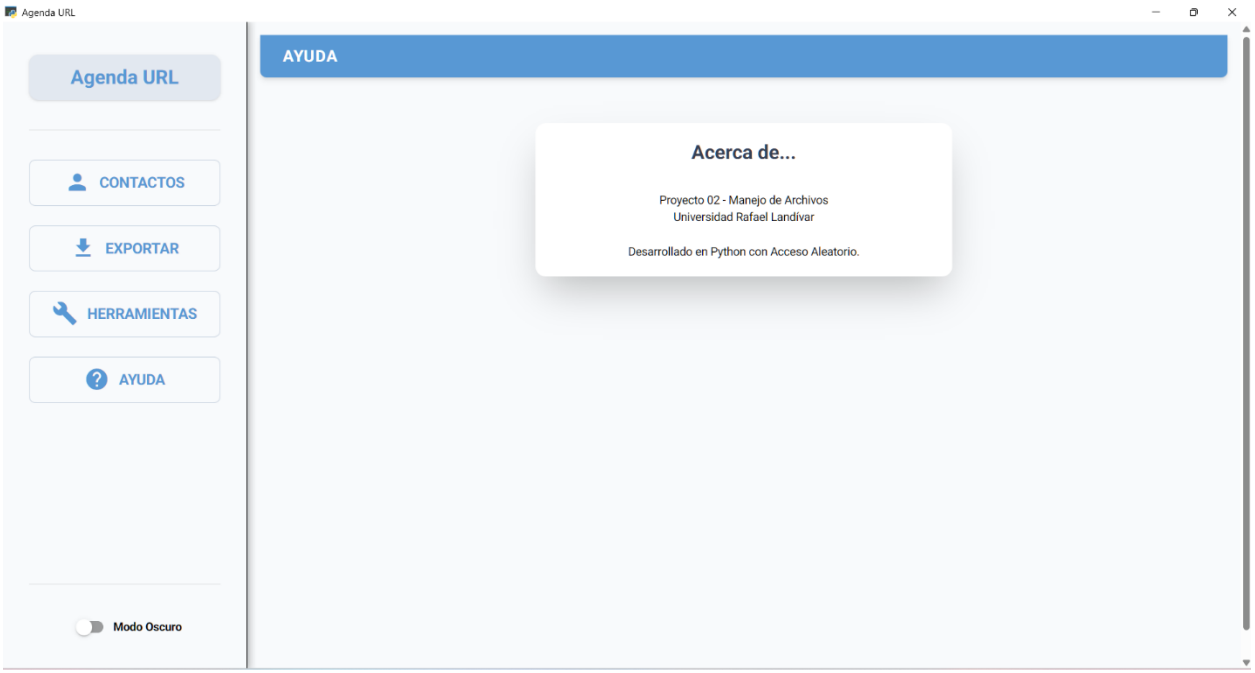





Agenda URL
Agenda URL
CONTACTOS
EXPORTAR
HERRAMIENTAS
AYUDA
Modo Oscuro
HERRAMIENTAS
INTEGRIDAD SHA-256
RESPALDO ZIP
METADATOS
Metadatos del Sistema
Consulta el tamaño exacto, fechas de modificación y permisos del sistema operativo.
VER METADATOS


Agenda URL
Agenda URL
CONTACTOS
EXPORTAR
HERRAMIENTAS
AYUDA
Modo Oscuro
HERRAMIENTAS
INTEGRIDAD SHA-256
RESPALDO ZIP
METADATOS
Metadatos del Sistema
Consulta el tamaño exacto, fechas de modificación y permisos del sistema operativo.
Metadatos del Archivo
Archivo: contactos.dat
Tamaño: 509 bytes
Última modificación: 2026-11-08 21:26:40
Permiso Lectura: Sí
Permiso Escritura: Sí
CERRAR

Opción de ayuda:



Opción de cambiar de tema (opción extra):

